```
!pip install -q transformers torchaudio ipywidgets
```

```
import torch
import torchaudio
from transformers import Wav2Vec2ForCTC, Wav2Vec2Tokenizer
from IPython.display import display
import ipywidgets as widgets
```

```
# Load tokenizer and model
tokenizer = Wav2Vec2Tokenizer.from_pretrained("jonatasgrosman/wav2vec2-large-xlsr-53-english")
model = Wav2Vec2ForCTC.from_pretrained("jonatasgrosman/wav2vec2-large-xlsr-53-english")

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
```

```
vocab.json: 100%  300/300 [00:00<00:00, 13.8kB/s]
special_tokens_map.json: 100%  85.0/85.0 [00:00<00:00, 4.42kB/s]
config.json:  1.53k/? [00:00<00:00, 55.8kB/s]
The tokenizer class you load from this checkpoint is not the same type as the class this function is called from. It may result
The tokenizer class you load from this checkpoint is 'Wav2Vec2CTCTokenizer'.
The class this function is called from is 'Wav2Vec2Tokenizer'.
/usr/local/lib/python3.12/dist-packages/transformers/models/wav2vec2/tokenization_wav2vec2.py:720: FutureWarning: The class `Wa
```

```
# File upload widget
upload = widgets.FileUpload(
    accept='.wav, .mp3',  # Accept WAV or MP3
    multiple=False
)

# Button to trigger transcription
transcribe_button = widgets.Button(
    description='Transcribe Audio',
    button_style='success'
)

# Output widget
output = widgets.Output()

# Function to transcribe audio
def transcribe_audio(b):
    with output:
        output.clear_output()
        if len(upload.value) == 0:
            print("Please upload an audio file first.")
            return

        # Get uploaded file
        audio_file = list(upload.value.values())[0]
        audio_bytes = audio_file['content']

        # Save temporary file
        with open("temp_audio.wav", "wb") as f:
            f.write(audio_bytes)

        # Load audio
        waveform, sample_rate = torchaudio.load("temp_audio.wav")

        # Resample if needed
        if sample_rate != 16000:
            waveform = torchaudio.transforms.Resample(orig_freq=sample_rate, new_freq=16000)(waveform)
            sample_rate = 16000

        # Tokenize
        input_values = tokenizer(waveform.squeeze().numpy(), return_tensors="pt").input_values.to(device)

        # Inference
        with torch.no_grad():
            logits = model(input_values).logits

        predicted_ids = torch.argmax(logits, dim=-1)
        transcription = tokenizer.decode(predicted_ids[0])

        print("Transcription:")
        print(transcription)

# Link button click
transcribe_button.on_click(transcribe_audio)

# Display form
display(upload, transcribe_button, output)
```

```
    )
      Upload (1)
  (dropout): Dropout(p=0.0, inplace=False)
    Transcribe Audio  (in_features=1024, out_features=33, bias=True)
)
Transcription:
the stale smell of old beer lingers it takes heat to bring out the odora cold dip restores health and zest a salt pickle taste
fine with ham tukozal pastor are my favorite a zestful food is the hot cross bun
```

```
Start coding or generate with AI.
```